Lecture 4: Nonlinearity and Risk Control

Ivy Yang

Economics & Finance, Business School, Southern Illinois University Edwardsville

1 From Stable Weights to Flexible Prediction Rules

1.1 The question inherited from Lectures 1–3

Lecture 1 established the target, forecast origin, horizon, permitted inputs and simple benchmark. Lecture 2 established chronological validation, overfitting and honest use of a final test. Lecture 3 asked how fitted weights and predictions change across nearby training samples, and used ridge and lasso to restrict coefficient sizes.

Lecture 4 changes the shape of the prediction rule. A rule may need to bend or use different responses in different settings. We first describe those shapes, then construct a decision tree, and finally combine trees in a random forest. The same information boundary, validation dates and comparison units still apply. A flexible model must earn its place through measured forecasting performance.

1.2 What risk control means here

Model risk is the possibility that a model gives misleading forecasts or is used inappropriately for the task. Here, risk control means limiting avoidable fitting sensitivity and checking the rule before relying on it. It does not mean that a forest guarantees small financial losses. Robustness means useful performance across the conditions or reasonable changes specified in an evaluation. It must be assessed for those conditions. A stable but biased prediction can still be wrong, and a new environment can invalidate an old relationship.

1.3 How to use this lecture with the notebook

Lecture 4 assumes the basic Python operations taught in the separate coding lessons for Lectures 1–3. There is no additional introductory coding recording for Lectures 4–6. The notebook is the detailed classroom companion: read the Markdown before each small code cell, run the worked example, inspect its output, then use the separate homework notebook for assigned tasks.

Every worked example has the same public number here, in the slides and in the notebook: Example 4.1, Example 4.2, and so on. Within the notebook, numbered steps such as 4.3.1 identify smaller operations. A separate homework task such as HW4.3 refers back to Example 4.3. Assignment instructions can cite these numbers directly. Python source files are

auxiliary reproducibility files; the student-facing lesson is the Jupyter notebook.

2 What Makes a Response Nonlinear?

2.1 A constant change versus a changing change

A linear rule without transformed inputs or interactions uses a constant fitted contribution per additional input unit, holding other inputs fixed. In the illustrative rule

$$\hat{y} = 10 + 2x,$$

x is an input, 10 is the starting level, 2 is the fitted weight, and $\hat{y}$ is the predicted outcome. Increasing x from 2 to 3 adds two outcome units. Increasing it from 7 to 8 also adds two.

Definition: Nonlinear response
The change in the prediction associated with one extra input unit depends on the input level or on another input. The graph can bend, change slope, or jump at a cutoff. This describes the fitted response; it does not establish a causal effect.

The curved rule in Example 4.1 uses the same one-unit input steps as the linear rule. Comparing its computed increments makes the changing response visible. Both rules are constructed illustrations; their predictions do not establish a causal business effect.

2.2 Polynomial terms: using powers of an input

A transformation constructs a new input value from existing input information. A polynomial term uses a power of an input, such as its square. A quadratic rule includes the squared input; its degree is two because two is the largest power used.

$$\hat{y} = 10 + 2x + x^2.$$

Here x^2 multiplies the input by itself. The three prediction contributions are the starting level, the ordinary-input contribution and the squared-input contribution. At $x = 2$, the prediction is $10 + 4 + 4 = 18$. At $x = 3$, it is $10 + 6 + 9 = 25$. The increase is seven. From $x = 3$ to $x = 4$, the increase is nine. The response bends even though the fitted coefficients enter additively.

A polynomial regression is linear in its coefficients: fitting still combines fixed input columns, such as x and x^2, with weights. Its response can be nonlinear in the original input. Adding powers increases flexibility and can also fit noise. Validation must compare candidate degrees on the same later cases. High powers can behave sharply outside the training input range.

2.2.1 Example 4.1: Constant and changing increments

A nonlinear response can change its increment as the input changes.

Step 1. Use inputs 2, 3, 4 for the quadratic rule and the separate one-unit linear pairs [2, 3] and [7, 8]. Keep the rules distinct; these are constructed predictions, not causal estimates.

Step 2. Calculate the squared column: 4, 9 and 16. Add the starting level and both contributions to obtain quadratic predictions 18, 25 and 34.

Step 3. Subtract adjacent predictions. The quadratic increments are 7 and 9 for equal input steps; the linear increments remain 2.

Step 4. Read the left graph as the two response shapes and the right graph as the three hand calculations. The transformed-column diagram shows that both x and its square enter the weighted sum.

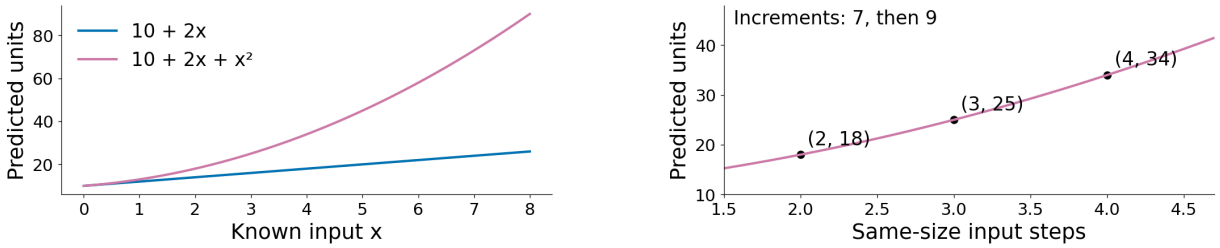


Figure 1: Example 4.1: Constant and changing increments. Computed or code-drawn by the matching numbered notebook example. Constructed hand examples and the classroom simulation are labeled separately.

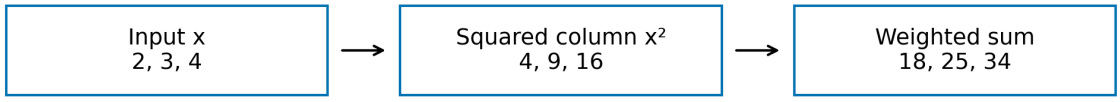


Figure 2: Example 4.1: Constant and changing increments. Computed or code-drawn by the matching numbered notebook example. Constructed hand examples and the classroom simulation are labeled separately.

Separate coding homework: HW4.1. Refer to lecture code 4.1; complete the blanks in the separate homework notebook.

2.3 Indicators, thresholds and regimes

A threshold is a specified cutoff. An indicator equals one when its stated condition is satisfied and zero otherwise. A regime is a setting in which the fitted response is allowed to differ. It can be defined by a measured condition; the word does not imply that we have discovered a permanent economic state.

Let $D = 1$ when a promotion is active and $D = 0$ otherwise. The condition must be known at the forecast origin if D is an input to a genuine forecast. A cutoff on an input is different from a decision threshold used by a manager after receiving a forecast.

2.3.1 Example 4.2: Turn a known condition into an indicator

An indicator is one when its stated condition holds and zero otherwise.

Step 1. Take a promotion schedule known at the forecast origin. The condition is whether promotion will be active; do not define it from the future sales outcome.

Step 2. Set D=0 for inactive and D=1 for active. The two values encode the two permitted settings.

Step 3. Check the displayed route: known schedule to indicator to a prediction input. The diagram is generated in code from those labels.

Step 4. Keep a threshold, an indicator and a regime distinct: the cutoff defines a condition, the indicator encodes its truth, and the regime names the setting.

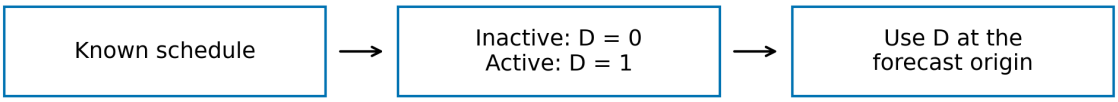


Figure 3: Example 4.2: Turn a known condition into an indicator. Computed or code-drawn by the matching numbered notebook example. Constructed hand examples and the classroom simulation are labeled separately.

Separate coding homework: HW4.2. Refer to lecture code 4.2; complete the blanks in the separate homework notebook.

2.4 Interactions: one input changes the response to another

Definition: Interaction

An interaction allows the fitted response to one input to depend on another input. Multiplying two inputs is one way to represent it.

Consider

$$\hat{y} = 10 + 2x + 5D + 3xD.$$

x is an input and D is the promotion indicator. The $5D$ block changes the starting level when the promotion is active. The $3xD$ block changes the input's fitted contribution. When $D = 0$, both promotion blocks disappear and the rule is $10 + 2x$. When $D = 1$, the rule is

$15 + 5x$. Thus an extra input unit adds two predicted units without promotion and five with promotion. All numbers are teaching illustrations, not estimated business effects.

2.4.1 Example 4.3: Calculate a promotion interaction

An interaction lets the response to x depend on another input.

Step 1. Use the same constructed rule everywhere: predicted y = 10 + 2x + 5D + 3xD. The input x and promotion D must already be known.

Step 2. Set x=2 and D=0. Both promotion terms vanish, leaving 10+4=14.

Step 3. Set x=2 and D=1. Add the contributions 10, 4, 5 and 6 to obtain 25. Over the full x grid, the two rules are 10+2x and 15+5x.

Step 4. Compare starting levels and slopes in the plotted lines. Switching D changes both in this example; it is not evidence of a causal promotion effect.

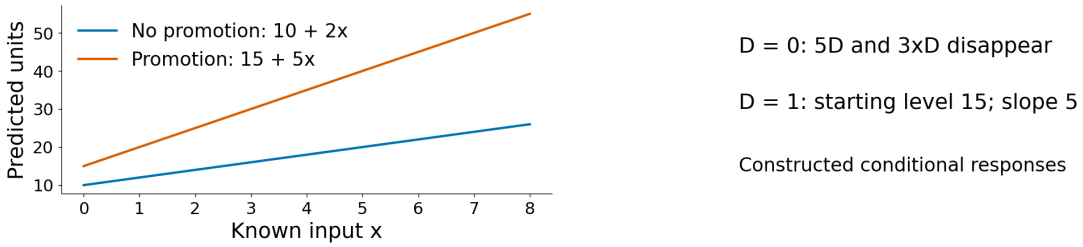


Figure 4: Example 4.3: Calculate a promotion interaction. Computed or code-drawn by the matching numbered notebook example. Constructed hand examples and the classroom simulation are labeled separately.

Separate coding homework: HW4.3. Refer to lecture code 4.3; complete the blanks in the separate homework notebook.

3 A Decision Tree Is a Sequence of Questions

3.1 The parts of a regression tree

Definition: Decision tree and regression tree

A decision tree routes an observation through fitted if–then questions. A regression tree predicts a numerical outcome, such as tomorrow's sales. In the squared-error trees used here, its prediction is the average training outcome among the observations reaching the same final group.

A node is a location in the tree. The root is the starting node containing all the training observations. A split divides a node's observations using an input and a cutoff. A branch

follows an answer to that question. A leaf is a terminal node: a final group with no further split.

A path is the sequence of answers followed by one case. Depth counts the splits along a path from the root. The root has depth zero; after one split, a case reaches depth one. Maximum depth is the largest permitted number of such splits along any path, not the number of trees and not the total number of leaves.

3.1.1 Example 4.4: Read roots, branches, leaves and depth

A regression tree routes a known input through questions to a stored numerical mean.

Step 1. Fit the eight traffic/sales pairs used in Example 4.5. The root contains eight observations and asks whether yesterday's traffic is at most 50.

Step 2. Follow Yes or No to the two leaves. A split is the question; branches are its answers; leaves are the terminal prediction groups.

Step 3. Fit a second tree with maximum depth two on the same pairs. Count splits along a case's path: root depth zero, first answer depth one, second answer depth two.

Step 4. Read the plotted fitted structure rather than counting leaves as depth. In the depth-two tree, the four stored means are 22, 30, 62 and 70.

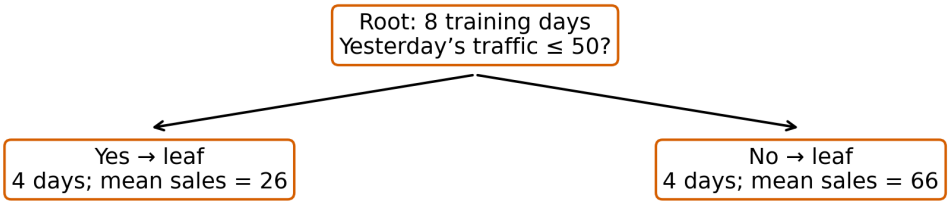


Figure 5: Example 4.4: Read roots, branches, leaves and depth. Computed or code-drawn by the matching numbered notebook example. Constructed hand examples and the classroom simulation are labeled separately.

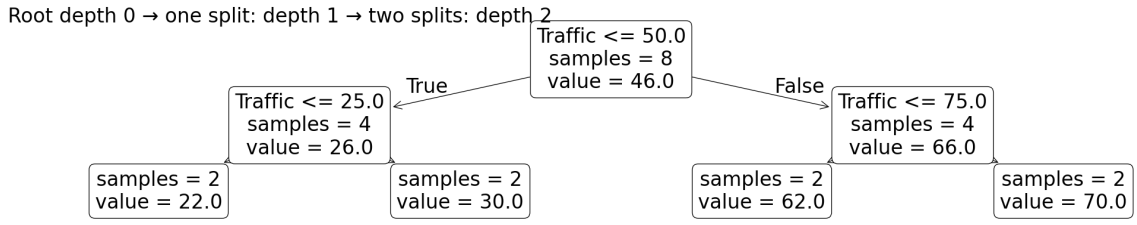


Figure 6: Example 4.4: Read roots, branches, leaves and depth. Computed or code-drawn by the matching numbered notebook example. Constructed hand examples and the classroom simulation are labeled separately.

Separate coding homework: HW4.4. Refer to lecture code 4.4; complete the blanks in the separate homework notebook.

3.2 A complete leaf prediction by hand

Suppose yesterday's observed customer traffic is the input and today's observed sales are the training outcome. Yesterday's traffic is available before today's sales. These eight illustrative training pairs are:

Yesterday's traffic	10	20	30	40	60	70	80	90
Today's sales	20	24	28	32	60	64	68	72

A candidate question asks whether traffic is at most 50. The left group contains sales 20, 24, 28 and 32, with average 26. The right group contains 60, 64, 68 and 72, with average 66. A new forecast case with yesterday's traffic of 35 is routed left and receives prediction 26. A case with traffic of 75 is routed right and receives 66. The new case is routed using its input; its future outcome is not needed to decide its path.

A piecewise constant rule uses a constant prediction inside each fitted region, with possible jumps at boundaries. This one-split tree is constant on either side of its threshold. It does not draw a sloped line between the observations inside a leaf.

3.2.1 Example 4.5: Compute leaf means and route new cases

A squared-error regression leaf predicts its training outcome mean.

Step 1. Pair traffic [10,20,30,40,60,70,80,90] with sales [20,24,28,32,60,64,68,72]. The input precedes the sales target.

Step 2. Split traffic at 50. Average the four left outcomes to get 26 and the four right outcomes to get 66.

Step 3. Route the new input 35 to the left and 75 to the right. The computed predictions are 26 and 66; the unknown new outcomes are never used in routing.

Step 4. Read the fitted graph as two constant regions, not a sloping interpolation within each leaf.

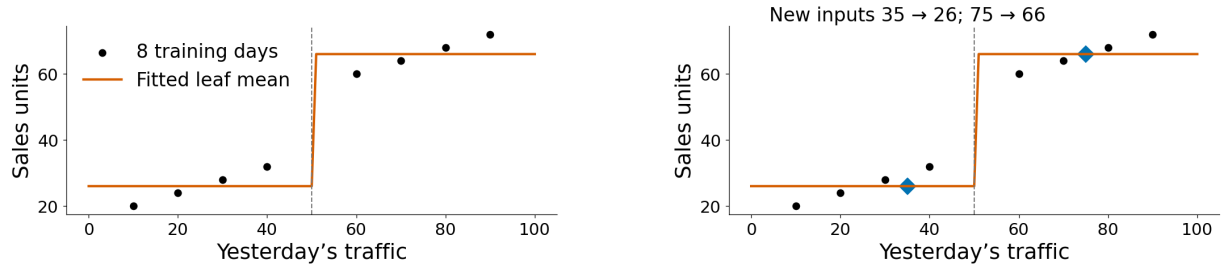


Figure 7: Example 4.5: Compute leaf means and route new cases. Computed or code-drawn by the matching numbered notebook example. Constructed hand examples and the classroom simulation are labeled separately.

Separate coding homework: HW4.5. Refer to lecture code 4.5; complete the blanks in the separate homework notebook.

3.3 How the fitted split is chosen

For the squared-error criterion, within-node variation means the sum of squared distances between each training outcome and its node's mean. Some software calls a node's average squared deviation its impurity. Here impurity describes outcome spread; it is not a judgment about data quality.

The single unsplit group has mean 46 and squared-error sum 3,360. Splitting at 50 gives squared-error sums 80 on the left and 80 on the right, for total 160. The reduction is 3,200. This is a training calculation, not a measured improvement in future forecast error.

The tree evaluates permitted inputs and cutoffs and chooses the best immediate training-error reduction at the current node. This is called a greedy procedure: it chooses the best current step rather than searching all possible complete trees. Minimum leaf size and other settings restrict the candidates. A locally best split need not lead to the globally best future tree.

3.3.1 Example 4.6: Choose a split by its immediate training gain

Greedy fitting chooses the best current training step, not an entire future tree.

Step 1. Keep the eight pairs fixed. Without splitting, the mean sales value is 46; squared distances from that mean sum to 3360.

Step 2. At cutoff 50, the two within-leaf squared-error sums are 80 and 80, giving total 160.

Step 3. Subtract 160 from 3360: the immediate training reduction is 3200. The candidate-cutoff table is computed for all seven separating cutoffs.

Step 4. Find the minimum of the plotted candidate scores. This explains the fitted cutoff 50 but does not measure validation improvement.

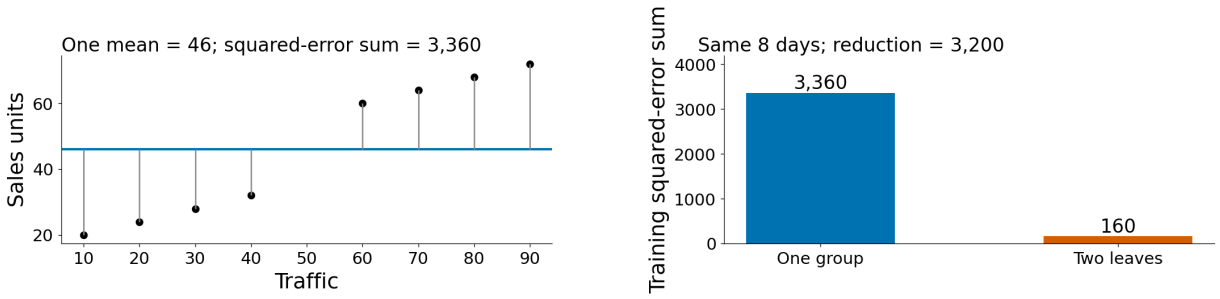


Figure 8: Example 4.6: Choose a split by its immediate training gain. Computed or code-drawn by the matching numbered notebook example. Constructed hand examples and the classroom simulation are labeled separately.

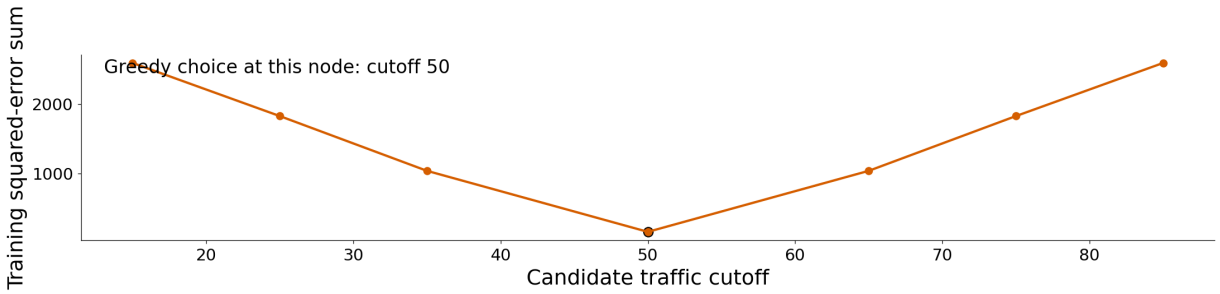


Figure 9: Example 4.6: Choose a split by its immediate training gain. Computed or code-drawn by the matching numbered notebook example. Constructed hand examples and the classroom simulation are labeled separately.

Separate coding homework: HW4.6. Refer to lecture code 4.6; complete the blanks in the separate homework notebook.

3.4 Rectangles and interactions in input space

Input space is a graph whose coordinates are input values. In two dimensions, one axis might be yesterday's traffic and the other the known promotion indicator. A partition divides that space into regions. Splitting on traffic draws a vertical boundary; splitting on the second input divides an existing region in the other direction. An axis-aligned split asks about one input coordinate at a time. Repeated one-input splits create rectangular regions in two dimensions.

If the promotion split is used only within the high-traffic branch, its contribution depends on traffic. The tree therefore represents an interaction without requiring an explicit product column. The displayed example uses exactly two inputs. Its colored regions and tree branches represent the same fitted groups.

3.4.1 Example 4.7: Connect rectangles to a conditional tree

One-input splits divide two-input space into rectangular prediction regions.

Step 1. Construct twelve cases from traffic [10,20,40,60,80,90] crossed with promotion D=0 or 1. This is a different labeled toy dataset from Example 4.3.

Step 2. Assign sales 20 when traffic is at most 50. Above 50, assign 30 without promotion and 50 with promotion.

Step 3. Fit the depth-two tree. It first separates traffic and then asks about promotion only inside the high-traffic branch.

Step 4. Predict [35,0], [35,1], [75,0] and [75,1]. The outputs 20,20,30,50 connect the rectangular colors to the tree's conditional routes.

Separate coding homework: HW4.7. Refer to lecture code 4.7; complete the blanks in the

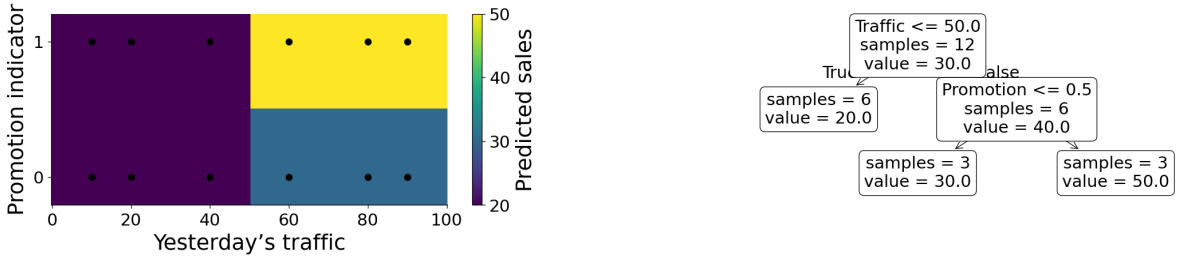


Figure 10: Example 4.7: Connect rectangles to a conditional tree. Computed or code-drawn by the matching numbered notebook example. Constructed hand examples and the classroom simulation are labeled separately.

separate homework notebook.

4 Restrict a Tree Before Trusting Its Detail

4.1 Depth, leaf size and overfitting

A deeper tree can create smaller groups and follow finer training patterns. If a final group contains one observation, its prediction can simply repeat that observation's outcome. New cases routed there inherit that potentially noisy value.

Minimum observations per leaf sets the smallest allowed final group. Pruning removes fitted branches, replacing their detail with a simpler prediction rule. Maximum depth, minimum leaf size and pruning are forms of regularization: they constrain complexity rather than adding a coefficient-size charge. Their settings are hyperparameters in the Lecture 3 sense.

For nested tree growth using the same criterion and sample, adding permitted splits cannot worsen the optimized training fit. Validation error has no corresponding guarantee. A more restrictive tree can underfit; an overly detailed tree can overfit. Increased restriction can reduce variation across fits but can introduce systematic error from an overly simple rule. The realized balance must be measured.

4.1.1 Run the classroom example rather than assume a curve

A simulation generates artificial observations from specified rules. The classroom snapshot contains 180 simulated daily demand values, with seed 417 recorded in its source. Two lagged-demand inputs leave 178 usable rows: 124 training days, 27 validation days and 27 final-test days. These are teaching data, not market evidence. The main comparison chooses depth 5 and minimum leaf size 4 by trying each permitted depth/leaf-size pair on validation. The one-control depth panel holds leaf size at two; it is not the complete joint selection search. The left panel compares depth; the middle panel varies minimum leaf size; the right panel shows how pruning strength changes the number of leaves. Each panel varies its stated control while keeping the data fixed.

4.1.2 Example 4.8: Prepare the classroom demand example

Simulation creates artificial observations from stated rules; these are not market data.

Step 1. Load the local business_demand.csv copied from the existing classroom package. It contains 180 simulated daily observations, source seed 417.

Step 2. Create demand one day and two days earlier. Removing the first two unavailable lags leaves 178 usable rows.

Step 3. Preserve date order. Use the first 124 rows for training, the next 27 for validation and the last 27 for final testing.

Step 4. Read the date-series graph and the shaded periods. Both predictors are observed before the current demand target; no download fallback changes the data source.

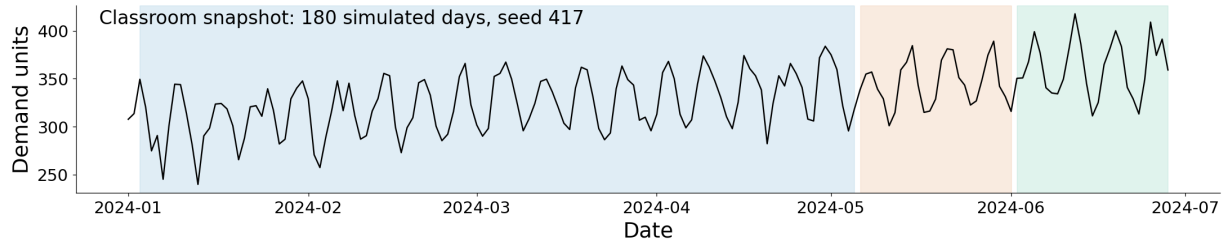


Figure 11: Example 4.8: Prepare the classroom demand example. Computed or code-drawn by the matching numbered notebook example. Constructed hand examples and the classroom simulation are labeled separately.

Separate coding homework: HW4.8. Refer to lecture code 4.8; complete the blanks in the separate homework notebook.

4.1.3 Example 4.9: Vary one tree control at a time

Depth, leaf size and pruning constrain different aspects of a tree.

Step 1. In the left panel, vary maximum depth from one to eight with minimum leaf size held at two. Compute training and validation RMSE on fixed dates.

Step 2. In the middle panel, vary minimum leaf size across 1,2,4,8,16 while maximum depth is eight.

Step 3. In the right panel, vary pruning strength across 0,5,20,80,200, with minimum leaf size two. Count the remaining leaves.

Step 4. These conditional comparisons explain the controls. They do not replace the joint depth/leaf-size search in Example 4.10.

Separate coding homework: HW4.9. Refer to lecture code 4.9; complete the blanks in the separate homework notebook.



Figure 12: Example 4.9: Vary one tree control at a time. Computed or code-drawn by the matching numbered notebook example. Constructed hand examples and the classroom simulation are labeled separately.

4.1.4 Example 4.10: Select depth and leaf size together

A grid tries a list of candidate settings; here every depth/leaf-size pair is compared.

Step 1. Cross eight maximum depths with five minimum leaf sizes, producing forty candidate pairs.

Step 2. Fit each candidate using the same 124 training rows and evaluate it on the same 27 validation rows.

Step 3. Select the pair with the smallest validation RMSE. This run chooses depth five and leaf size four, with validation RMSE about 20.63.

Step 4. Read the marked heatmap cell, then carry that pair to refitting. The final test is not used to select it.



Figure 13: Example 4.10: Select depth and leaf size together. Computed or code-drawn by the matching numbered notebook example. Constructed hand examples and the classroom simulation are labeled separately.

Separate coding homework: HW4.10. Refer to lecture code 4.10; complete the blanks in the separate homework notebook.

4.2 Why a fitted tree can be sensitive

A slightly different sample can change the best split, and an early split changes which observations are available to later splits. Even if the question stays unchanged, its leaf average can move. In the right leaf above, replacing sales 60 with 100 raises its average from 66 to 76. The input 75 now receives a prediction ten units higher. This is an explicit sensitivity illustration with one changed outcome, not evidence that every tree is unstable by exactly ten units.

Compare predictions on common reference inputs across refits, as in Lecture 3. Comparing different dates with different inputs confounds changes in the fitted rule with changes in the cases. A smooth-looking forecast path is not, by itself, a repeated-sample stability calculation.

4.2.1 Example 4.11: Change one leaf while fixing the query

A model-sensitivity comparison keeps the prediction case fixed.

Step 1. Keep the right-leaf outcomes 60,64,68,72 and the traffic threshold 50. Their mean is 66.

Step 2. Replace only the first outcome, 60, with 100. The right-leaf mean becomes 76.

Step 3. Query the same input 75 before and after the replacement. The prediction rises by ten without changing the case.

Step 4. This holds the split fixed to isolate the leaf average. It is not a claim that a newly optimized full tree must retain the same split.

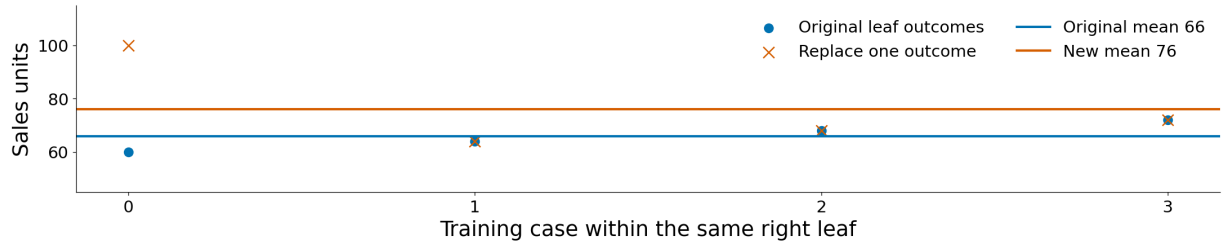


Figure 14: Example 4.11: Change one leaf while fixing the query. Computed or code-drawn by the matching numbered notebook example. Constructed hand examples and the classroom simulation are labeled separately.

Separate coding homework: HW4.11. Refer to lecture code 4.11; complete the blanks in the separate homework notebook.

5 From One Tree to a Random Forest

5.1 An ensemble and the meaning of the name

An ensemble combines predictions from several fitted models. Its individual models are called base learners; here they are trees. A random forest is an ensemble of trees built with deliberate random variation, whose numerical predictions are averaged. "Forest" names the collection of trees. "Random" refers to variation introduced during their construction, not random selection of the final answer.

5.1.1 Example 4.12: Expand one model into a forest

An ensemble combines fitted models; a random forest averages varied fitted trees.

Step 1. Identify one decision-tree diagram as one base learner, using the node and leaf meanings established in Example 4.4.

Step 2. Expand it into a collection of seven schematic trees. Different structures represent possible fitted rules; seven is a drawing choice, not a required forest size.

Step 3. Send the same known input case through all member rules rather than comparing different cases.

Step 4. Combine the numerical predictions by their mean. The next examples show how rows and candidate inputs vary and how the average is computed.

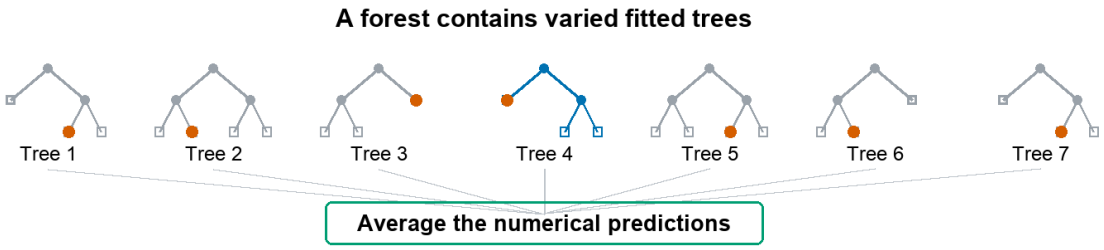


Figure 15: Example 4.12: Expand one model into a forest. Computed or code-drawn by the matching numbered notebook example. Constructed hand examples and the classroom simulation are labeled separately.

Separate coding homework: HW4.12. Refer to lecture code 4.12; complete the blanks in the separate homework notebook.

5.2 Bootstrap sampling and input randomization

Lecture 2 introduced bootstrap resampling. For an ordinary tree bootstrap, draw training row identifiers with replacement. With replacement means a selected row remains eligible to be drawn again. One tree might receive identifiers A, A, C and D from a four-row training pool; another might receive A, B, B and D. Some rows repeat and some are absent in each draw.

At a split, a forest can also restrict the search to a randomly selected subset of input columns. This is feature subsampling. It varies which inputs may compete for that split; it does not mean that the outcome is hidden from the training fit. If every column is permitted, this particular source of diversity disappears. Record the actual software setting rather than assuming random input subsampling from the model name.

A random seed initializes a reproducible sequence of random choices. Fixing it helps reproduce a fit. Repeating the same seed on the same sample does not measure sensitivity to new training data.

5.2.1 Example 4.13: Draw training rows with replacement

With replacement means a selected row remains eligible to be drawn again.

Step 1. Start both draws from the same training pool A,B,C,D. The second tree does not draw from the first tree's bag.

Step 2. The first constructed draw is A,A,C,D: A repeats and B is absent.

Step 3. The second draw is A,B,B,D: B repeats and C is absent. Each bag still contains four selected rows.

Step 4. The branching arrows show parallel resampling from the original pool. Every row must belong to the permitted training history.

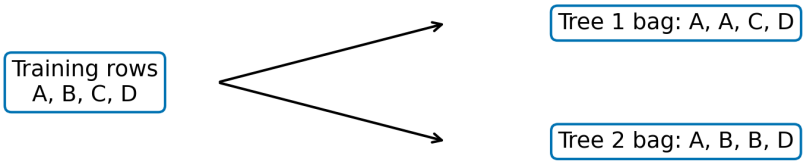


Figure 16: Example 4.13: Draw training rows with replacement. Computed or code-drawn by the matching numbered notebook example. Constructed hand examples and the classroom simulation are labeled separately.

Separate coding homework: HW4.13. Refer to lecture code 4.13; complete the blanks in the separate homework notebook.

5.2.2 Example 4.14: Inspect candidate feature draws

Feature subsampling restricts the input columns that may compete at a split.

Step 1. Use four illustrative input names: Traffic, Advertising, Weather and Promotion.

Step 2. With seed 417, draw two distinct candidate columns at each of three illustrative nodes. The generated pairs are Promotion/Traffic, Advertising/Promotion and Promotion/Advertising.

Step 3. The third pair repeats the second set in a different order. Random draws may repeat a subset; different draws need not produce different sets.

Step 4. A fixed seed reproduces these choices. It does not test sensitivity to a different training sample.

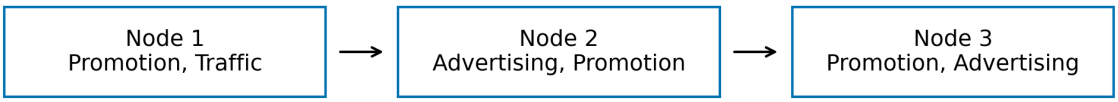


Figure 17: Example 4.14: Inspect candidate feature draws. Computed or code-drawn by the matching numbered notebook example. Constructed hand examples and the classroom simulation are labeled separately.

Separate coding homework: HW4.14. Refer to lecture code 4.14; complete the blanks in the separate homework notebook.

5.3 Averaging predictions for the same case

Let B be the number of trees. Let x denote the input values for one prediction case, and let $\hat{f}_b(x)$ be tree b's prediction for that case. The tree label b runs from one to B. The forest prediction is

$$\hat{y}_{\text{forest}}(x) = \frac{1}{B}\sum_{b=1}^{B} \hat{f}_b(x).$$

Each summand is one tree's numerical prediction. The sum collects them, and division by the tree count gives their arithmetic mean. Example 4.15 fits three depth-one trees to stated resampled bags and obtains predictions 50, 60 and 70 for the same query, giving average 60. For numerical regression this is averaging values, not a majority vote among class labels. Classification predicts a category rather than the numerical outcome studied here.

5.3.1 Example 4.15: Fit three trees and average one shared query

A regression forest averages numerical member predictions for the same case.

Step 1. Use a six-row constructed pool: identifiers L,A,B,C,D,E; inputs [-1,1,1,1,1,1]; outcomes [10,40,50,60,70,80].

Step 2. Fit three depth-one trees to bags [L,A,A,B,C,C], [L,A,B,C,D,E] and [L,C,C,D,E,E]. Each learned root splits x at zero.

Step 3. Query x=1. The right-leaf outcome means are 50,60,70. These are predictions of the actual fitted trees, not numbers attached to unrelated tree pictures.

Step 4. Add 50+60+70 and divide by three to get 60. The left query x=-1 would return 10

from every tree.

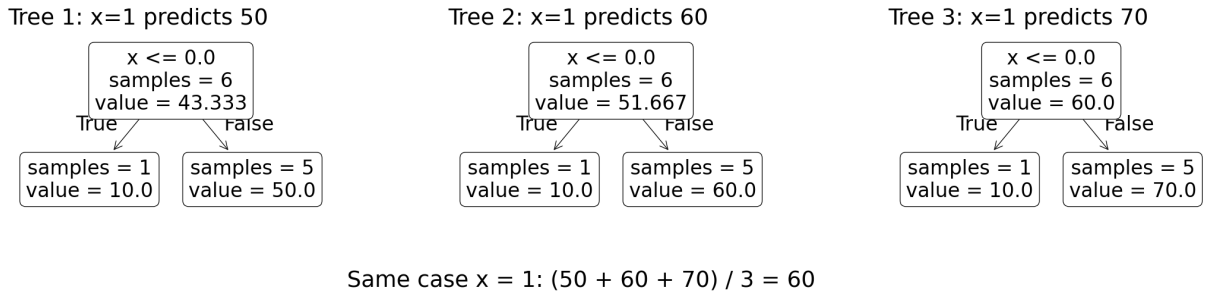


Figure 18: Example 4.15: Fit three trees and average one shared query. Computed or code-drawn by the matching numbered notebook example. Constructed hand examples and the classroom simulation are labeled separately.

Separate coding homework: HW4.15. Refer to lecture code 4.15; complete the blanks in the separate homework notebook.

5.4 Why different errors matter

Averaging can offset tree-specific errors when their deviations differ. If three trees miss the same fixed target by $+6$, -3 and -3, their average error is zero in that illustration. If all miss by $+6$, the average also misses by $+6$. The second case shows a shared error that averaging cannot remove.

Error correlation describes how strongly different trees' errors move together across comparable repeated cases or samples. Less-similar errors leave more opportunity for averaging to reduce variance. Bootstrap variation and feature subsampling encourage diversity, but trees still share data and can make common mistakes. They are not independent just because their fitting uses randomness.

More trees can make the randomized average settle more reliably for a fixed fitting procedure. They do not supply new training observations, automatically remove bias, or guarantee lower future RMSE. The variation among trees for one fitted forest is also not automatically a calibrated prediction interval, meaning an interval whose claimed coverage is supported by evaluation on later outcomes.

5.4.1 Example 4.16: Compare offsetting and shared errors

Averaging can offset differing errors but cannot remove an identical shared error.

Step 1. Hold the target case fixed. Construct three errors +6,-3,-3, expressed as prediction minus target.

Step 2. Calculate their mean: zero. The plotted average marker falls on the zero-error reference.

Step 3. Now change the error vector to +6,+6,+6. Its mean remains +6; the average marker stays on the same side as every member.

Step 4. These are controlled illustrations of cancellation and shared error, not a proof that a forest achieves a given accuracy.

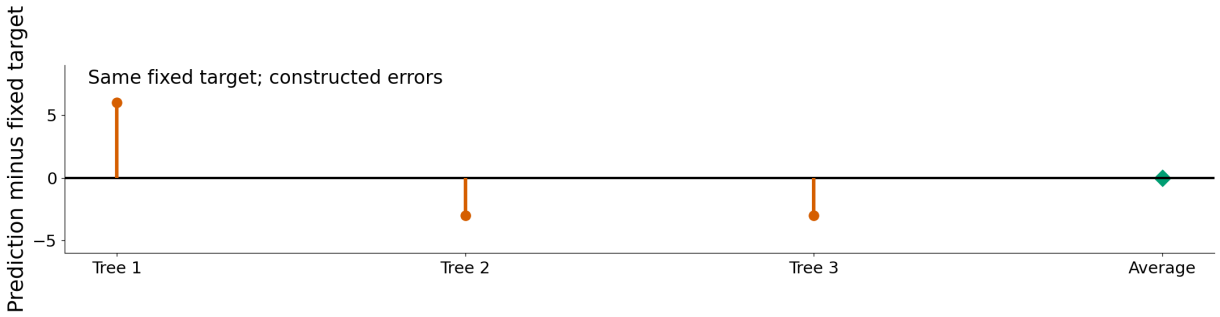


Figure 19: Example 4.16: Compare offsetting and shared errors. Computed or code-drawn by the matching numbered notebook example. Constructed hand examples and the classroom simulation are labeled separately.

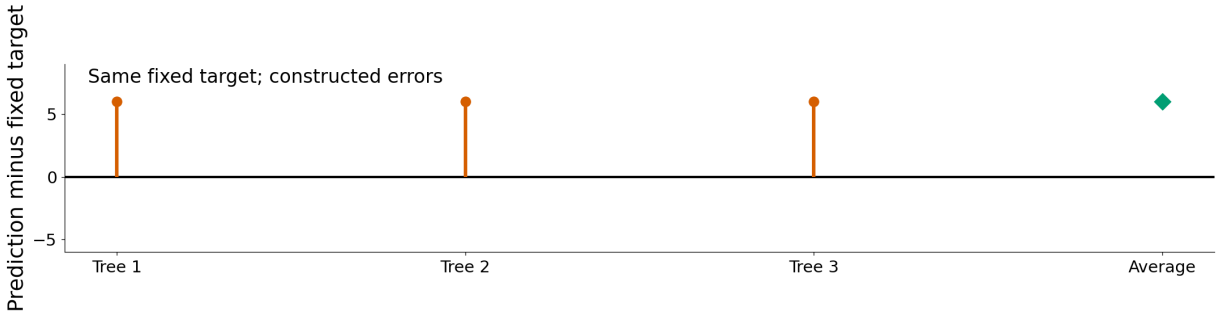


Figure 20: Example 4.16: Compare offsetting and shared errors. Computed or code-drawn by the matching numbered notebook example. Constructed hand examples and the classroom simulation are labeled separately.

Separate coding homework: HW4.16. Refer to lecture code 4.16; complete the blanks in the separate homework notebook.

5.5 Time order still applies inside a forest workflow

Select tree depth, leaf size, feature-subset settings and other choices using later validation periods, with the test untouched. Resampling for trees must draw only from that fold's permitted training history. The randomization used to fit the model does not authorize a random train/test split across dates. Ordinary bootstrap draws do not preserve local time dependence; use chronological held-out evaluation even if the learner uses such draws internally. Lecture 2's block-bootstrap uncertainty procedure has a different purpose from creating diverse trees.

5.5.1 Example 4.17: Keep the forest inside chronological training

Random tree construction does not authorize future rows in the training history.

Step 1. Reuse the chronological blocks from Example 4.8: 124 training, 27 validation and 27 test rows.

Step 2. Permit internal bootstrap draws only from the current training block. Feature-subset settings concern columns, not permission to move future dates into training.

Step 3. Use validation to choose development settings and retain the final test for the settled procedure.

Step 4. Distinguish the learner's row resampling from the block-bootstrap uncertainty calculation introduced in Lecture 2.

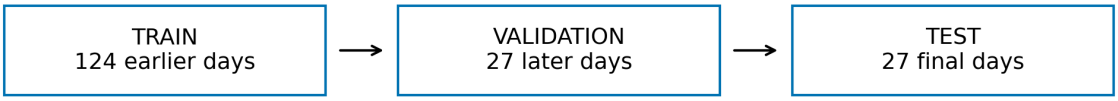


Figure 21: Example 4.17: Keep the forest inside chronological training. Computed or code-drawn by the matching numbered notebook example. Constructed hand examples and the classroom simulation are labeled separately.

Separate coding homework: HW4.17. Refer to lecture code 4.17; complete the blanks in the separate homework notebook.

6 Understand What an Importance Score Measures

6.1 Training split importance

Definition: Feature importance

An input's importance is its use under a stated model-scoring procedure. The meaning depends on the procedure; there is no universal importance number attached to a variable.

Impurity-based importance credits an input for reductions in within-node training variation when splits use that input, weighted by node size and combined across trees. Software commonly normalizes those credits to sum to one when the total reduction is positive. Example 4.18 calculates three constructed credits 60, 30 and 10. Example 4.19 separately reports the measured two-input classroom forest shares, approximately 0.699 and 0.301. These are different labeled examples, not interchangeable data. These are shares of a training

split score, not percentages of future accuracy or causal effects. A forest with no effective splits may have all-zero importances.

Inputs offering many possible cutoffs can receive favorable training scores merely because they offer more split opportunities. Related predictors can substitute for one another and divide the score. These are reasons to inspect held-out behavior rather than to treat the ranking as a conclusion about economic mechanisms.

6.1.1 Example 4.18: Normalize three split-credit totals

Normalizing divides each credit by a positive total so the shares sum to one.

Step 1. Use the constructed raw credits 60,30,10 for three illustrative inputs. This hand calculation is separate from the two-input classroom forest.

Step 2. Add the credits: the total is 100.

Step 3. Divide each credit by 100 to obtain 0.60,0.30,0.10, and check that the shares sum to one.

Step 4. Read these as shares of the chosen training split score, not percentages of future accuracy.

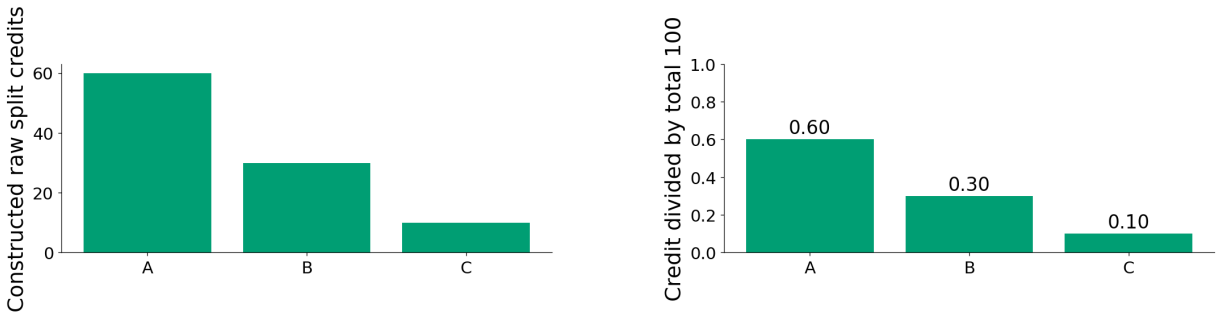


Figure 22: Example 4.18: Normalize three split-credit totals. Computed or code-drawn by the matching numbered notebook example. Constructed hand examples and the classroom simulation are labeled separately.

Separate coding homework: HW4.18. Refer to lecture code 4.18; complete the blanks in the separate homework notebook.

6.1.2 Example 4.19: Read the fitted forest importance

An importance number has meaning only under its stated scoring procedure.

Step 1. Fit the classroom forest on the 124 training rows with 100 trees, depth five, minimum leaf size eight, one candidate input per split and seed 417.

Step 2. Read the fitted training split-credit shares for the two lagged-demand inputs. They are approximately 0.699 and 0.301.

Step 3. Check the total of the positive normalized shares. The two-input score is not the separate three-input normalization example.

Step 4. Keep training split importance distinct from permutation importance: one credits training splits; the other changes input alignment and reevaluates a fixed model.

Step 5. Minimum split size is the number of observations a node must contain before a split may be attempted. As a separate edge case, minimum split size 200 exceeds the 124 training rows. The code fits a forest that cannot make effective splits; its two importance entries are both zero.



Figure 23: Example 4.19: Read the fitted forest importance. Computed or code-drawn by the matching numbered notebook example. Constructed hand examples and the classroom simulation are labeled separately.

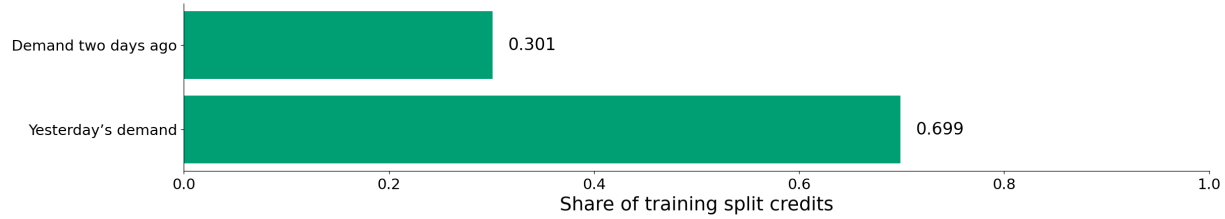


Figure 24: Example 4.19: Read the fitted forest importance. Computed or code-drawn by the matching numbered notebook example. Constructed hand examples and the classroom simulation are labeled separately.

Separate coding homework: HW4.19. Refer to lecture code 4.19; complete the blanks in the separate homework notebook.

6.2 Permutation importance

A permutation rearranges a list without changing its entries. For permutation importance, first measure the fitted model's error on a specified evaluation sample. Rearrange one input's values across the cases, keep the model fixed, and measure the error again. This deliberately breaks that column's original connection to the outcomes and other inputs.

In the classroom calculation of Example 4.20, the fixed forest has validation RMSE about 23.33. The twenty-shuffle means are approximately 32.72 for the one-day lag and 26.27 for the two-day lag, increases of about 9.39 and 2.93 demand units. Repeat the rearrangement to see how sensitive that calculation is to the random shuffle. If another input contains

similar information, the shuffled input can look unimportant because the model still has a substitute. With time-indexed data, unrestricted shuffling can also create unrealistic temporal combinations. State the rearrangement scheme and the dependence it breaks; use it as a diagnostic, not a simulation of an actual market intervention.

If importance results guide feature selection or model revision, calculate and use them within the development/validation process. Using the final test to revise the inputs turns that test into development data. The final test is reserved for evaluating the settled procedure.

6.2.1 Example 4.20: Shuffle one validation input at a time

Permutation importance reevaluates a fixed model after an input is rearranged.

Step 1. Keep the trained classroom forest and all 27 validation outcomes fixed. Its unshuffled validation RMSE is about 23.33.

Step 2. Rearrange one input column, leaving its values intact but changing their row alignment. The left panel displays a five-row illustration, not all evaluation rows.

Step 3. Repeat twenty rearrangements per input over the full validation block. Mean RMSE is approximately 32.72 for the one-day lag shuffle and 26.27 for the two-day lag shuffle.

Step 4. Compare those errors with the same unshuffled value. The increases are approximately 9.39 and 2.93 demand units; these are development diagnostics.

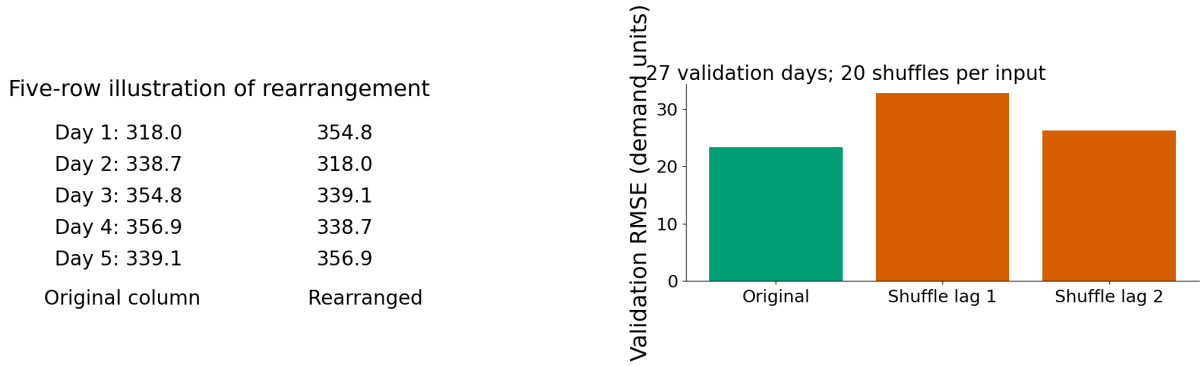


Figure 25: Example 4.20: Shuffle one validation input at a time. Computed or code-drawn by the matching numbered notebook example. Constructed hand examples and the classroom simulation are labeled separately.

Separate coding homework: HW4.20. Refer to lecture code 4.20; complete the blanks in the separate homework notebook.

6.2.2 Example 4.21: Trace substitute information through two columns

Related predictors can supply substitute information to a fitted model.

Step 1. Construct twenty counter values and copy the same values into a second column. The two traces deliberately overlap.

Step 2. Rearranging one column leaves the other column's values and ordering intact.

Step 3. Identify exactly which information connection the perturbation breaks and which substitute remains.

Step 4. The plot illustrates the information structure. It is not itself a measured claim about a trained model's permutation score; any such score must be computed and labeled separately.

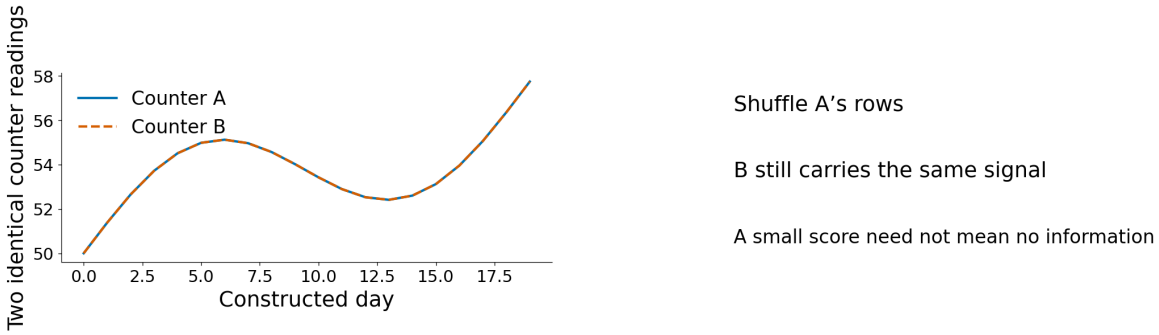


Figure 26: Example 4.21: Trace substitute information through two columns. Computed or code-drawn by the matching numbered notebook example. Constructed hand examples and the classroom simulation are labeled separately.

Separate coding homework: HW4.21. Refer to lecture code 4.21; complete the blanks in the separate homework notebook.

7 An Honest Comparison and an Operating Decision

7.1 Compare a whole procedure

A pipeline is the full ordered sequence that turns raw permitted data into a prediction: data preparation, any learned transformations and the fitted model. Fit learned preparation using the permitted training block only. Compare the simple benchmark, the restricted linear model, the tree and the forest on the same target, horizon, origins, cases and error units.

For ordinary axis-aligned trees, a rescaling that preserves the order of input values also preserves the corresponding split groups in exact arithmetic. Example 4.22 compares traffic reported in customers and in hundreds of customers. Scaling is therefore not the same requirement it was for coefficient penalties in Lecture 3. Missing-value handling, transformations or feature selection can still learn from data and must obey the information boundary.

7.1.1 Example 4.22: Preserve a pipeline and compare reporting units

A pipeline applies ordered preparation and model steps; changing units can preserve tree groups.

Step 1. Fit any learned preparation and the prediction rule using permitted earlier observations. Apply the stored procedure to later inputs.

Step 2. Reuse the eight traffic/sales pairs. Fit once with traffic in customers, and once with traffic divided by 100, reported in hundreds of customers.

Step 3. The learned cutoffs are 50 customers and approximately 0.5 hundreds. Queries 35 and 75 become 0.35 and 0.75.

Step 4. Both fitted trees give predictions 26 and 66. Floating-point cutoff display can differ slightly, but the fitted group assignments and predictions agree for these cases.

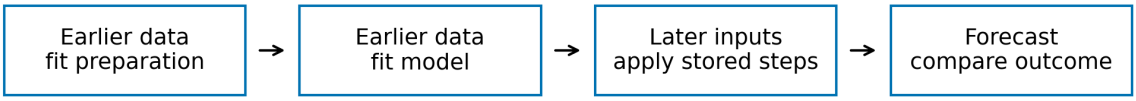


Figure 27: Example 4.22: Preserve a pipeline and compare reporting units. Computed or code-drawn by the matching numbered notebook example. Constructed hand examples and the classroom simulation are labeled separately.

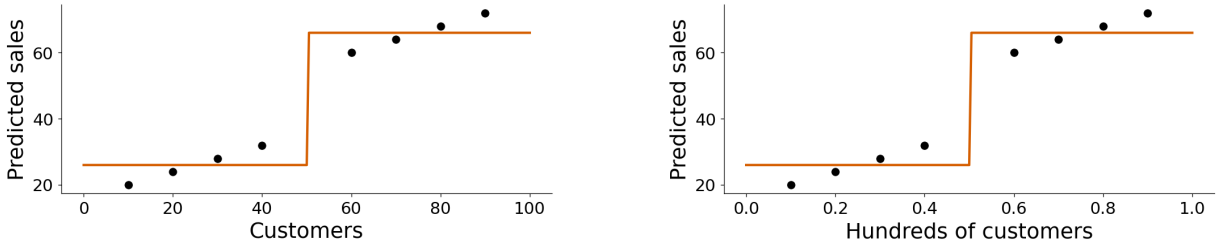


Figure 28: Example 4.22: Preserve a pipeline and compare reporting units. Computed or code-drawn by the matching numbered notebook example. Constructed hand examples and the classroom simulation are labeled separately.

Separate coding homework: HW4.22. Refer to lecture code 4.22; complete the blanks in the separate homework notebook.

7.2 A measured classroom comparison

The following scores are produced by the accompanying code from the local synthetic demand snapshot. Validation chooses the depth and ridge strength. Each settled candidate is then refitted using training plus validation before the final test. The two lagged inputs,

chronological evaluation dates and demand units are shared; persistence uses the most recent demand directly.

Candidate	Validation RMSE	Final-test RMSE
Linear	20.62	28.77
Quadratic	21.09	29.11
Ridge	20.62	28.77
Tree	20.63	30.90
Forest	23.33	32.55
Persistence	22.89	29.12

Linear regression has the smallest validation RMSE in this run, although ridge and the jointly selected tree are extremely close on validation. The forest's greater flexibility does not beat it here. A decision based on validation therefore does not require selecting a nonlinear model. The final-test scores evaluate settled procedures; they are not another tuning round. These results describe this simulation and implementation, not a universal ranking of methods.

A forest may require explaining and maintaining many fitted routes even when the raw input count is the same. If another task shows a forecasting gain, state that gain and its operating cost. Monitoring means checking later errors, input availability, missing values and the relevance of the fitted rule after it is put into use. Keep the input definitions, fitting periods, settings and revision history so another analyst can reproduce and inspect the comparison.

7.2.1 Example 4.23: Report validation and refitted test results

Model comparison uses the same target, dates and units, with settings chosen before final testing.

Step 1. Compare Linear, Quadratic, Ridge, the jointly selected Tree, the preset Forest and Persistence on the same validation dates.

Step 2. The validation RMSE values are approximately 20.62, 21.09, 20.62, 20.63, 23.33, 22.89. Linear, ridge and tree are very close; a forest win is not assumed.

Step 3. Refit each settled learned procedure using training plus validation. Then predict the final 27 dates; persistence uses the latest observed demand directly.

Step 4. Report the test scores without retuning: approximately 28.77, 29.11, 28.77, 30.90, 32.55, 29.12 in the same model order. These results concern this simulation, not a universal ranking.

Separate coding homework: HW4.23. Refer to lecture code 4.23; complete the blanks in the separate homework notebook.

7.2.2 Example 4.24: Refit while holding reference inputs fixed

Changing the training sample and changing the prediction case answer different questions.

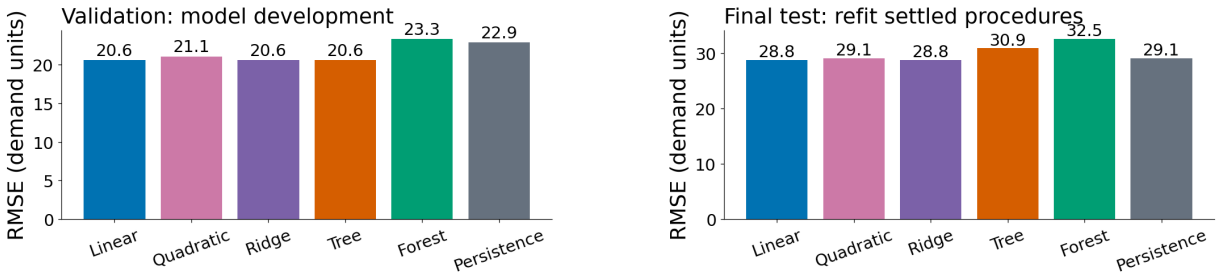


Figure 29: Example 4.23: Report validation and refitted test results. Computed or code-drawn by the matching numbered notebook example. Constructed hand examples and the classroom simulation are labeled separately.

Step 1. Construct low-lag and high-lag reference input pairs using each training input's 25th and 75th percentiles. A percentile is a position in the ordered values: approximately 25 percent or 75 percent lie at or below those positions. Keep both vectors unchanged throughout the diagnostic.

Step 2. Fit the same selected tree and preset forest on six 80-day windows ending at rows 80,90,100,110,120,124 within training history.

Step 3. After each fit, predict the two unchanged vectors and plot each vector in its own panel.

Step 4. Variation along a panel tracks refit sensitivity for a fixed case. A difference between panels reflects different input conditions and must not be called model instability.

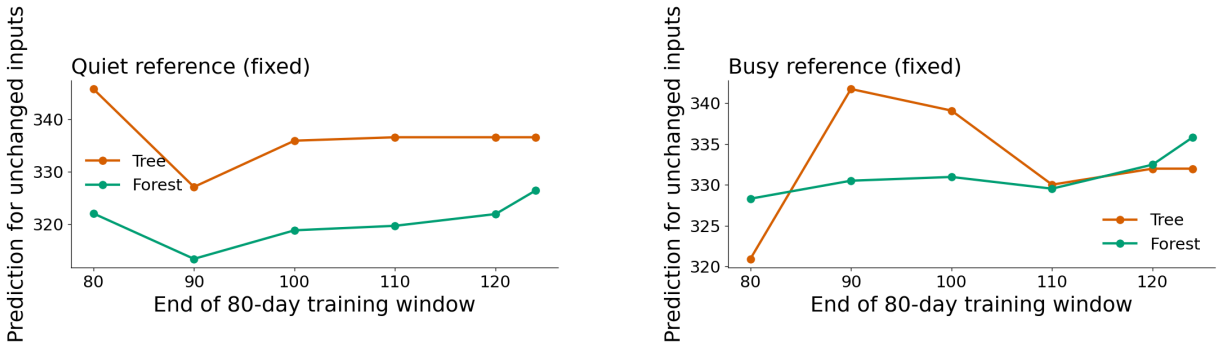


Figure 30: Example 4.24: Refit while holding reference inputs fixed. Computed or code-drawn by the matching numbered notebook example. Constructed hand examples and the classroom simulation are labeled separately.

Separate coding homework: HW4.24. Refer to lecture code 4.24; complete the blanks in the separate homework notebook.

7.3 Limits that the next lecture will examine

Extrapolation means applying the fitted rule outside the input conditions represented in training. In a standard regression tree with mean-valued leaves, a new extreme input still receives one of the stored leaf averages. An average of such trees cannot generate a fitted prediction outside the minimum and maximum of the training outcomes used across those trees. This limits automatic continuation of a trend beyond observed experience.

A forest can learn a complex relationship inside the observed setting and still fail after regime change or under a different decision cost. Lecture 5 develops those limits and the connection from a prediction to a decision. Lecture 4 establishes how the nonlinear rule is built and evaluated; it does not turn flexibility into a guarantee.

7.3.1 Example 4.25: Predict beyond the observed input range

Extrapolation applies a rule outside represented input conditions.

Step 1. Reuse the same eight traffic/sales training pairs, whose largest observed input is 90.

Step 2. Fit the one-split tree and linear regression using only those pairs.

Step 3. Query inputs out to 140. The tree keeps returning its right-leaf mean 66; the fitted linear rule continues its line.

Step 4. Read the shaded region as outside the training input range. No target outcomes in that region were supplied to validate either continuation.

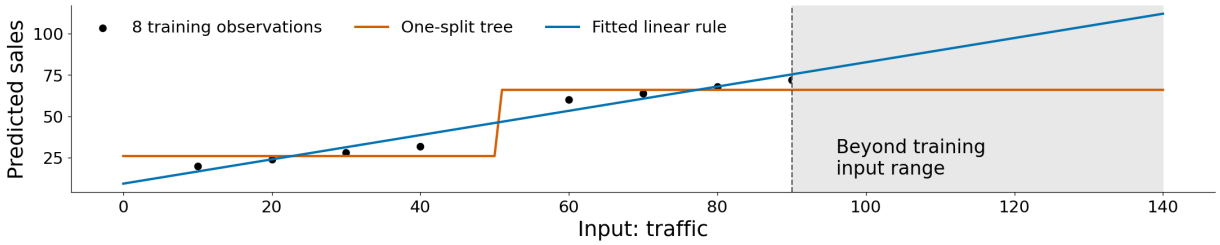


Figure 31: Example 4.25: Predict beyond the observed input range. Computed or code-drawn by the matching numbered notebook example. Constructed hand examples and the classroom simulation are labeled separately.

Separate coding homework: HW4.25. Refer to lecture code 4.25; complete the blanks in the separate homework notebook.

8 Checks You Should Be Able to Complete

1. Explain the difference between a changing response to x and a rule that remains linear in its fitted coefficients.

2. Set the promotion indicator to zero and one, and show why the illustrative slopes are

two and five.

3. Route inputs 35 and 75 through the one-split tree; compute the two leaf averages.

4. Explain why a reduction in training squared error is different from a validation improvement.

5. Describe depth, minimum leaf size, pruning, bootstrap samples, feature subsampling and the tree-count average.

6. Explain why averaging different errors can help and why averaging identical errors cannot.

7. Distinguish training split importance, held-out permutation importance and a causal effect.

8. Design a repeated-fit stability check using common reference inputs and an untouched chronological test.

9 Connection to the Coding Exercise

For assignments, cite the public example number and its practice suffix; the notebook table of contents links to every example. The current executable notebook is Code/Lecture 4 nonlinear_methods.ipynb. Its matching source is Lecture 4/Coding Part/lecture4_examples.py. Run All executes the examples, saves the figures used in these slides and writes numerical results to Coding Part/outputs/. The local demand CSV is copied from the existing classroom preparation package. It is always labeled simulated; there is no silent download fallback.

The notebook first reproduces the hand examples, then fits actual trees and forests, compares chronological validation scores, refits settled candidates before the test, and measures common-case sensitivity. The earlier financial notebook is retained in the archive as a reference. Its same-day shape plots are descriptive, and its changing-case rolling spread should not be read as common-case refit stability.

10 Technical References

The tree and forest mechanics follow the scikit-learn documentation: Decision trees, Forest ensembles, and Permutation importance. Numerical business examples in these notes are explicitly constructed teaching examples.